

# Preprocessing in NLP

## 1 Overview

Before feeding text into an NLP model, a series of preprocessing steps is performed. The common pipeline is as follows:

1. **Text Normalization**
2. **Stemming / Lemmatization**
3. **Tokenization (or Subword Tokenization)**
4. **Embedding**

Each step builds on the previous one to reduce variability, remove noise, and standardize the data for effective learning.

## 2 Why This Order?

- **Text Normalization First:** Normalization cleans the raw text by removing inconsistencies (e.g., punctuation, typos, and case differences) and standardizing the text. This ensures that the subsequent steps operate on a cleaner input, reducing vocabulary size and minimizing noise.
- **Stemming / Lemmatization Next:** After normalization, stemming or lemmatization is applied to further reduce word variability by mapping different inflected forms to a common base form. Doing this after normalization guarantees that the words are in a consistent format before reducing them.
- **Tokenization:** Once the words have been normalized and reduced to their canonical forms, tokenization splits the text into tokens (words or subwords) that will be used to build a vocabulary.
- **Embedding:** Finally, tokens are converted into numerical vectors (embeddings) that serve as input to downstream models.

## 3 Detailed Explanations of Each Stage

### 3.1 Text Normalization

**Purpose:** To clean and standardize the text, reducing variability and noise.

**Techniques:**

- **Lowercasing:** Convert all text to lowercase (e.g., “Apple” and “apple” become identical).
- **Punctuation Removal:** Remove or separate punctuation marks (e.g., converting “hello!” to “hello”).
- **Whitespace Standardization:** Replace multiple spaces, newlines, or tabs with a single space.
- **Accent and Diacritic Removal:** Convert accented characters to their unaccented forms (e.g., “résumé” becomes “resume”).
- **Expanding Contractions:** Replace contractions with their full forms (e.g., “don’t” becomes “do not”).
- **Spelling Correction:** Fix common typos and misspellings.
- **Special Character Handling:** Normalize or remove symbols such as emoji or HTML tags.
- **Number Normalization:** Standardize the representation of numbers.

**Reason:** Normalization is performed first so that all subsequent steps operate on a consistent and cleaner text, reducing the chance that minor variations cause different tokens.

### 3.2 Stemming and Lemmatization

**Purpose:** To reduce words to a base form so that different forms of the same word are treated identically.

#### Stemming

**Method:** Uses heuristic rules to chop off prefixes and suffixes. For example, the Porter Stemmer applies a series of rules to remove common endings.

##### Pros:

- Fast and simple.
- Reduces vocabulary size.

##### Cons:

- May produce non-dictionary forms.
- Can be overly aggressive, conflating distinct words.

#### Lemmatization

**Method:** Uses linguistic rules and dictionaries (often with POS tagging) to convert words to their dictionary (lemma) form. For example, “running,” “ran,” and “runs” all become “run,” and “better” may be lemmatized to “good” when used as an adjective.

##### Pros:

- Produces valid words.
- Better preserves the meaning of the word.

**Cons:**

- More computationally intensive.
- Dependent on accurate POS tagging and language-specific resources.

**Reason:** Applying stemming or lemmatization after normalization ensures that the text is in a consistent format, reducing the vocabulary further and making tokenization more effective.

### 3.3 Tokenization (and Subword Tokenization)

**Purpose:** To split text into smaller units (tokens) that serve as the basic units for analysis.

**Techniques:**

- **Word Tokenization:** Splits text based on spaces and punctuation.
- **Subword Tokenization:** Uses methods such as Byte Pair Encoding (BPE), WordPiece, or SentencePiece to split words into subword units. This is useful for handling rare or out-of-vocabulary words.

**Reason:** Tokenization follows stemming/lemmatization because, once the words have been standardized, splitting them into tokens results in a consistent vocabulary suitable for the model.

### 3.4 Embedding

**Purpose:** To map tokens to numerical vectors that capture semantic and syntactic information.

**Techniques:**

- **Pre-trained Word Embeddings:** Such as Word2Vec, GloVe, or fastText.
- **Contextual Embeddings:** Derived from models like BERT or GPT that provide dynamic representations based on context.

**Reason:** After tokenization, converting tokens to embeddings provides the numerical representation required for downstream tasks (e.g., classification, translation) using neural networks or other machine learning models.

## 4 Summary

The overall preprocessing pipeline for NLP typically follows this order:

1. **Normalization:** Clean and standardize the text.
2. **Stemming / Lemmatization:** Reduce words to their base or canonical forms.
3. **Tokenization:** Split text into tokens or subword units.
4. **Embedding:** Convert tokens to numerical vectors for model input.

This order ensures that each step builds on a progressively cleaner and more consistent representation of the text, ultimately making it more amenable to analysis and learning in downstream NLP tasks.